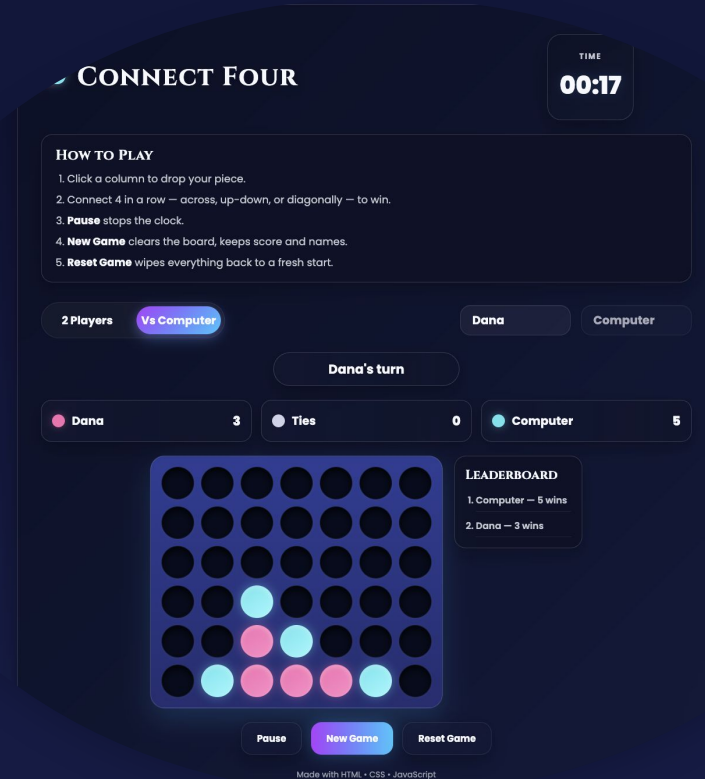


CONNECT FOUR

A browser-based classic, **built completely from scratch** —
no React, no frameworks — just HTML, CSS & JavaScript.

Dana

General Assembly · Software Engineering Bootcamp Capstone



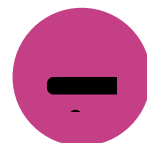
What I Built & Why

No React. No frameworks. Just HTML, CSS, and JavaScript — plus a small **confetti** effect for the win.



A 2D Grid, Built by Hand

A 6×7 board rendered and fully managed in the DOM — no framework doing the diffing.



Turn-Based State

Tracking whose turn it is, which moves are legal, and when the game has ended.



A Computer Opponent

A lightweight AI that plays tactically instead of dropping pieces at random.



Persistent Scores

Scores and the all-time leaderboard survive a refresh, via localStorage.

WALKTHROUGH

Live Demo

What I'll show, in order:

1 Start Game — locks the board until you're ready — a real kickoff

3 Pause & Resume — board dims, clock freezes

5 Win + Confetti — winning line highlighted on the board

7 Light / Dark Theme — one click, full re-theme

2 Alternating Turns — status pill tracks whose move it is

4 Vs Computer — switch on the tactical AI opponent

6 Scoreboard & Leaderboard — saved to localStorage, survives a refresh

8 Responsive Layout — desktop down to a phone screen

How It's Built

Two functions worth a closer look — open `js/script.js` live.

script.js — `findWinningCells()`

```
function findWinningCells(row, column, player) {
  for (const [rowStep, colStep] of DIRECTIONS) {
    const line = [[row, column],
      ...getLine(row, column, rowStep, colStep,
        player),
      ...getLine(row, column, -rowStep, -colStep,
        player)];
    if (line.length >= 4) return line;
  }
  return [];
}
```

Walk outward in all 4 directions from the placed piece — 4+ connected in a line, and it returns exactly those cells to highlight.

script.js — `getComputerColumn()`

```
function getComputerColumn() {
  const valid = openColumns();

  const findTacticalMove = (player) =>
    valid.find((column) => {
      place(column, player);           // try it
      const wins = findWinningCells(...);
      undo(column);                     // then undo
      return wins.length > 0;
    });

  return findTacticalMove('Player2')   // win
    ?? findTacticalMove('Player1')     // block
    ?? random(valid);                  // else
  Tests every open column by placing a piece, checking for a win, then undoing
  it — win if possible, block if needed, otherwise random.
}
```

A Few Reflections

4

Your favorite part?

The computer opponent — getting a simple 'test every column, undo it' loop to feel like it's actually playing with intent. Watching it block me for the first time was a real payoff.

5

Most challenging part?

Getting the win-detection logic right across all 4 directions without double-counting — and getting the AI's tactical check to test a move without permanently mutating the board state.

6

What did you learn?

How much state management you have to handle by hand without a framework — whose turn it is, what's legal, when the game ends, what needs to persist. Real appreciation for what React does under the hood.

Thank You

Questions & feedback welcome.



Live Demo dana12812.github.io/connect4-game



Source Code github.com/dana12812/connect4-game

